

Median of Two Sorted Arrays

Yesterday 11:16 PM

For **Median of Two Sorted Arrays**, there are several approaches, but the important one for interviews and optimal complexity is **binary search on the smaller array**.

LeetCode 4 is designed around this idea.

1. First understand the problem

Suppose:

```
A = [1, 3, 8]
B = [2, 7, 10, 12]
```

Combined sorted array would be:

```
[1, 2, 3, 7, 8, 10, 12]
```

Median:

```
7
```

But we **cannot actually merge** the arrays if we want the optimal solution.

The goal is:

```
O(log(min(n, m)))
```

instead of:

```
O(n + m)
```

2. Approaches

Approach 1 — Merge completely

Use two pointers like merge sort:

```
A = [1,3,8]
B = [2,7,10,12]
→ [1,2,3,7,8,10,12]
```

Time:

```
O(n + m)
```

Space:

```
O(n + m)
```

We can optimize space, but still need $O(n+m)$ time.

Approach 2 — Two pointers without actually storing merged array

We only move until the middle.

Time:

```
O((n+m)/2)
```

Still approximately:

```
O(n+m)
```

Better space, but not optimal.

Approach 3 — Binary Search Partition ★

This is the important approach.

Time:

```
O(log(min(n,m)))
```

Space:

```
O(1)
```

3. The key idea behind binary search

This problem becomes much easier if you stop thinking:

"How do I find the median?"

and instead think:

"How can I divide the two arrays into a left half and a right half?"

For example:

```
A = [1, 3, 8]
B = [2, 7, 10, 12]
```

Total elements:

```
3 + 4 = 7
```

We want:

```
Left half | Right half
```

Since there are 7 elements:

```
4 elements | 3 elements
```

One valid partition is:

```
A = [1, 3 | 8]
B = [2, 7 | 10, 12]

      ↑
    partition
```

Left side:

```
[1,3,2,7]
```

Right side:

```
[8,10,12]
```

The important property is:

```
every element on left <= every element on right
```

So we need:

```
max(left) <= min(right)
```

4. How do we find this partition?

Suppose:

```
A = [1, 3, 8]
B = [2, 7, 10, 12]
```

Let's partition A after 2 elements:

```
A = [1, 3 | 8]
```

Then the number of elements needed on the left from B is:

```
totalLeft - elementsFromA
```

Total elements:

```
7
```

Left side should contain:

```
(7 + 1) / 2 = 4
```

So:

```
elementsFromB = 4 - 2
               = 2
```

Therefore:

```
A = [1, 3 | 8]
B = [2, 7 | 10, 12]
```

5. Four important values

After partitioning, we only care about **four elements**.

```

      left      right
A    [1, 3] | [8]
      |
      LA_right
B    [2, 7] | [10, 12]
      |
      LB_right

```

Define:

```
Aleft = A[i-1]
Aright = A[i]
Bleft = B[j-1]
Bright = B[j]
```

For this example:

```
Aleft = 3
Aright = 8

Bleft = 7
Bright = 10
```

6. What makes a partition correct?

We need:

```
Aleft <= Bright
```

AND

```
Bleft <= Aright
```

Why?

Because the largest element on the left must not be greater than the smallest element on the right.

So:

```
max(Aleft, Bleft) <= min(Aright, Bright)
```

is the core condition.

7. Why binary search works

Suppose:

```
A = [1, 3, 8]
B = [2, 7, 10, 12]
```

We choose a partition in A.

Let:

```
i = number of elements taken from A
```

Then:

```
j = half - i
```

Now check:

```
Aleft <= Bright
Bleft <= Aright
```

There are three possibilities.

Case 1 — Correct partition

```
Aleft <= Bright
AND
Bleft <= Aright
```

We found the answer.

Case 2 — Too many elements taken from A

Suppose:

```
Aleft > Bright
```

Example:

```
Aleft = 10
Bright = 7
```

That means:

```
10
 \
  > 7
```

We have put something too large into the left side.

Therefore we need to move the partition in **A left**.

```
i--
```

Binary search:

```
high = i - 1
```

Case 3 — Too few elements taken from A

Suppose:

```
Bleft > Aright
```

For example:

```
Bleft = 10
Aright = 7
```

We need to move more elements from A into the left side.

Therefore:

```
i++
```

Binary search:

```
low = i + 1
```

That's the entire binary-search logic.

8. Why search the smaller array?

This is extremely important.

Suppose:

```
A has 1,000,000 elements
B has 10 elements
```

We should binary search:

```
B
```

because complexity becomes:

```
O(log 10)
```

rather than:

```
O(log 1,000,000)
```

So first:

```
C++
if (A.size() > B.size())
    swap(A, B);
```

Now:

```
A = smaller array
B = larger array
```

9. The complete algorithm

Let:

```
n = A.size()
m = B.size()
```

We want the left side to contain:

```
half = (n + m + 1) / 2
```

Binary search over:

```
i = 0 ... n
```

Then:

```
j = half - i
```

Now calculate:

```
Aleft = i == 0 ? -∞ : A[i-1]
Aright = i == n ? +∞ : A[i]

Bleft = j == 0 ? -∞ : B[j-1]
Bright = j == m ? +∞ : B[j]
```

Then:

```
if (Aleft <= Bright && Bleft <= Aright)
```

partition is correct.

10. Finding the median

Once the partition is correct:

```
maxLeft = max(Aleft, Bleft)
minRight = min(Aright, Bright)
```

If total number of elements is odd

Example:

7 elements

Median:

maxLeft

So:

```
C++
return max(Aleft, Bleft);
```

If total number is even

Example:

8 elements

There are two middle elements:

maxLeft
minRight

Therefore:

```
C++
return (maxLeft + minRight) / 2.0;
```

11. Complete C++ implementation

```
C++
class Solution {
public:
    double findMedianSortedArrays(vector<int>& A, vector<int>& B) {

        // Always binary search on the smaller array
        if (A.size() > B.size()) {
            swap(A, B);
        }

        int n = A.size();
        int m = B.size();

        int low = 0;
        int high = n;

        int half = (n + m + 1) / 2;

        while (low <= high) {
            // Partition A
            int i = low + (high - low) / 2;

            // Partition B
            int j = half - i;

            int Aleft = (i == 0) ? INT_MIN : A[i - 1];
            int Aright = (i == n) ? INT_MAX : A[i];

            int Bleft = (j == 0) ? INT_MIN : B[j - 1];
            int Bright = (j == m) ? INT_MAX : B[j];

            // Correct partition
            if (Aleft <= Bright && Bleft <= Aright) {
                // Odd total length
                if ((n + m) % 2 == 1) {
                    return max(Aleft, Bleft);
                }

                // Even total length
                return (max(Aleft, Bleft) +
                        min(Aright, Bright)) / 2.0;
            }

            // Too many elements taken from A
            else if (Aleft > Bright) {
                high = i - 1;
            }

            // Too few elements taken from A
            else {
                low = i + 1;
            }
        }

        return 0.0;
    }
};
```

12. Dry run

Take:

```
A = [1, 3]
B = [2, 4, 5, 6]
```

Total:

```
6
```

So:

```
half = (6 + 1) / 2
      = 3
```

We binary search A.

Initially:

```
low = 0
high = 2
```

Iteration 1

```
i = 1
```

Therefore:

```
j = 3 - 1
  = 2
```

Partitions:

```
A = [1 | 3]
B = [2,4 | 5,6]
```

Four values:

```
Aleft = 1
Aright = 3

Bleft = 4
Bright = 5
```

Check:

```
Aleft <= Bright
1 <= 5      ✓

Bleft <= Aright
4 <= 3      ✗
```

So partition is wrong.

We need more elements from A on the left.

```
low = i + 1
    = 2
```

Iteration 2

```
i = 2
```

Then:

```
j = 3 - 2
  = 1
```

Partitions:

```
A = [1,3 | ]
B = [2 | 4,5,6]
```

Values:

```
Aleft = 3
Aright = +∞
```

```
Bleft = 2
Bright = 4
```

Check:

```
3 <= 4 ✓
2 <= +∞ ✓
```

Correct partition.

So:

```
left = [1,3,2]
right = [4,5,6]
```

Since total is even:

```
median = (max(3,2) + min(∞,4)) / 2
        = (3 + 4) / 2
        = 3.5
```

Correct.

13. The mental model you should remember

Don't memorize the code.

Remember this picture:

```

      partition
      ↓
A = [ ..... LEFT ..... | ..... RIGHT ..... ]
B = [ ..... LEFT ..... | ..... RIGHT ..... ]
      ↓
    LEFT has half elements
      ↓
max(left) <= min(right)
      ↓
    median

```

And the four values:

```

Aleft | Aright
-----
Bleft | Bright

```

The condition is:

```
Aleft <= Bright
Bleft <= Aright
```

If:

```
Aleft > Bright
```

move A partition **left**.

If:

```
Bleft > Aright
```

move A partition **right**.

That's the binary search.

Complexity

Approach	Time	Space
Merge arrays	$O(n+m)$	$O(n+m)$

Approach	Time	Space
Two pointers	$O(n+m)$	$O(1)$
Binary search partition	$O(\log(\min(n,m)))$	$O(1)$

The **binary-search partition approach** is the one you should master for this problem because it teaches a very important pattern:

Binary search doesn't always search for an element. It can search for the correct partition/position satisfying a mathematical condition.

Today 10:29 AM

Yes. This is the **most important part** of the whole binary-search solution. If you understand these two conditions practically, the entire algorithm becomes much easier.

Let's forget the code for a moment and visualize what the partition is trying to achieve.

1. What are we actually trying to do?

Suppose:

```
A = [1, 3, 8]
B = [2, 7, 10, 12]
```

We want to divide both arrays into:

```
      LEFT | RIGHT
A = [1, 3, | 8]
B = [2, 7, | 10, 12]
```

So globally:

```
LEFT  = [1, 3, 2, 7]
RIGHT = [8, 10, 12]
```

We don't care that **LEFT** itself isn't sorted.

What we **do** care about is:

```
Every element in LEFT <= every element in RIGHT
```

In other words:

```
      LEFT      RIGHT
      1         8
      2         10
      3         12
      7  <=
           12
```

If this is true, then we have successfully divided the combined sorted array into two halves.

2. Why do we only check 4 elements?

Because both arrays are already sorted.

Consider:

```
A = [1, 3 | 8]
B = [2, 7 | 10, 12]
```

The largest element on the left can only be:

```
Aleft = 3
Bleft = 7
```

Therefore:

```
max(left) = max(Aleft, Bleft)
           = 7
```

Similarly, the smallest element on the right can only be:

```
Aright = 8
Bright = 10
```

Therefore:

```
min(right) = min(Aright, Bright)
            = 8
```

So we only need to verify:

```
7 <= 8
```

That's why the four values are enough.

3. Now understand this condition

```
C++
Aleft <= Bright && Bleft <= Aright
```

It means:

```
Aleft <= Bright
```

AND

```
Bleft <= Aright
```

Why exactly these two?

Let's draw it.

```
A = [ ..... Aleft | Aright ..... ]
B = [ ..... Bleft | Bright ..... ]
```

There are two possible **cross-boundary problems**.

Problem 1

Could **Aleft** be bigger than **Bright**?

```
Aleft > Bright
```

If yes:

A LEFT	B RIGHT
10	7
10 > 7 ❌	

Then something on the left is bigger than something on the right.

So our partition is wrong.

Therefore:

```
C++
Aleft <= Bright
```

must be true.

Problem 2

Could **Bleft** be bigger than **Aright**?

```
Bleft > Aright
```

Example:

A RIGHT	B LEFT
5	9
9 > 5 ❌	

Again, something on the left is bigger than something on the right.

So:

```
C++
Bleft <= Aright
```

must also be true.

4. So what does the condition really mean?

This:

```
C++
Aleft <= Bright && Bleft <= Aright
```

really means:

"There is no element on the left side that is greater than an element on the right side."

That's the practical meaning.

Or even simpler:

"The partition is in the correct position."

5. Now let's understand Aleft > Bright

This is the second part you specifically asked about:

```
C++
else if (Aleft > Bright)
```

Suppose we have:

```
A = [1, 3, 8]
B = [2, 7, 10, 12]
```

But we accidentally choose this partition:

```
A = [1, 3, 8 | ]
B = [2, 7 | 10, 12]
```

So:

```
Aleft = 8
Aright = ∞

Bleft = 7
Bright = 10
```

Now check:

```
Aleft <= Bright ?
8 <= 10
YES
```

and:

```
Bleft <= Aright ?
7 <= ∞
YES
```

Actually this partition is valid for the **left-half size 5**, but our required half is 4, so this example doesn't correspond to the same **half**. Let's use the binary-search logic with the correct half to see the actual failure.

For the same arrays, suppose binary search picks:

```
i = 2
j = 2
```

Partition:

```
A = [1, 3 | 8]
B = [2, 7 | 10, 12]
```

This is actually correct:

```
Aleft = 3
Aright = 8

Bleft = 7
Bright = 10
```

So:

```
3 <= 10 ✓
7 <= 8 ✓
```

Now imagine we moved the partition of A one position too far right in a different situation:

```
A = [1, 5, 9 | 20]
B = [2, 3, 6, 7, 10]
```

Suppose:

```
Aleft = 9
Bright = 7
```

Then:

```
9 > 7
```

So:

```
C++
Aleft > Bright
```

means:

We have taken too many elements from A into the left half.

Because this large value 9 should not be on the left while 7 is still on the right.

6. What should we do then?

We need to move the partition of A **toward the left**.

Before:

```
A = [1, 5, 9 | 20]
      ↑
    partition
```

Problem:

```
9 > 7
```

So move partition left:

```
A = [1, 5 | 9, 20]
      ↑
```

Now 9 goes to the right side.

This fixes the problem.

That's why we do:

```
C++
high = i - 1;
```

Because binary search needs to search the **left portion**.

7. The opposite situation

Now consider:

```
C++
Bleft > Aright
```

Example:

```
A = [1, 5 | 9, 20]
B = [2, 3, 6 | 7, 10]
```

Here:

```
Bleft = 6
Aright = 9
```

That's actually fine.

Let's make the problem:

```
A = [1, 3 | 4, 20]
B = [2, 8, 10 | 12]
```

Then:

```
Bleft = 10
Aright = 4
```

So:

```
10 > 4 ❌
```

We have:

LEFT		RIGHT
... 10		4 ...
↑		↑
Bleft		Aright

That's obviously wrong.

We need to move **more elements from A into the left side**.

Current:

```
A = [1, 3 | 4, 20]
```

Move partition right:

```
A = [1, 3, 4 | 20]
```

Therefore:

```
C++
low = i + 1;
```

8. So the entire binary search can be understood like this

Imagine the partition of A as a wall:

```
A = [ LEFT | RIGHT ]
      ↑
    WALL
```

We are moving this wall.

Wall is too far RIGHT

```
A = [1, 3, 8 | ...]
```

We have too many A elements on the left.

Detected by:

```
C++
Aleft > Bright
```

So:

```
← move wall
```

Code:

```
C++
high = i - 1;
```

Wall is too far LEFT

```
A = [ 1 | 3, 8, ... ]
```

We don't have enough A elements on the left.

Detected by:

```
C++
Bleft > Aright
```

So:

```
move wall →
```

Code:

```
C++
low = i + 1;
```

9. The most important visualization

Remember this:

```

      A partition
      ↓
A = [ A-left | A-right ]
      B partition
      ↓
B = [ B-left | B-right ]
```

We want:

```

      EVERYTHING HERE
      ↓
A-left  B-left
  \      /
   LEFT
   |
   | boundary
   |
  RIGHT
  /      \
A-right  B-right
```

For this boundary to be correct:

```
Aleft <= Bright
Bleft <= Aright
```

That's it.

10. Why don't we check Aleft <= Aright?

You might wonder:

Why don't we check `Aleft <= Aright`?

Because A is already sorted.

So:

```
Aleft <= Aright
```

is automatically true.

Similarly, because B is sorted:

```
Bleft <= Bright
```

is automatically true.

Therefore, the only possible problems are **between the two arrays**:

```

Aleft <= Bright
Bleft <= Aright

```

That's why exactly these two comparisons are used.

11. Think of it like merging two sorted arrays

Imagine you actually merged:

```

A = [1, 3, 8]
B = [2, 7, 10, 12]

```

You would get:

```

[1, 2, 3, 7 | 8, 10, 12]
      ↑
    median boundary

```

The binary-search algorithm is basically saying:

"I don't want to perform the merge. I just want to find where this | should be."

And these conditions tell us whether the | is correctly positioned.

```

Aleft <= Bright
Bleft <= Aright

```

If true:

```

FOUND IT ✓

```

If:

```

Aleft > Bright

```

then:

```

partition A is too far right
      ↓
    move A left

```

If:

```

Bleft > Aright

```

then:

```

partition A is too far left
      ↓
    move A right

```

The one-line intuition

Aleft > Bright means "I took too many elements from A into the left half, so move A's partition left."

Bleft > Aright means "I took too few elements from A into the left half, so move A's partition right."

Once you see the partition as a **wall that must be moved left/right**, the binary search becomes much easier to remember.

ChatGPT can make mistakes. Check important info.